

Universidade Federal do Amazonas (UFAM)

Instituto de Computação (ICOMP)

Relatório – Trabalho 2

Algoritmos e Estruturas de Dados 2 (AED2)

Árvores Binárias de Pesquisa e Árvore AVL

Alunos:

Alonso Ramos Brito Neto

Adrielly Silva de Souza

Gabriel Conceição dos Santos

Pedro Vinicius Weil Montenegro II

Manaus – AM
2026

Sumário

1	Introdução e Metodologia	2
2	Questão 1: Árvore Binária com Caminhamentos	2
2.1	Enunciado	2
2.2	Detalhes de Implementação	2
2.3	Código-fonte	3
2.4	Análise	4
3	Questão 2: Simulação de Pacotes de Rede	4
3.1	Enunciado	4
3.2	Detalhes de Implementação	4
3.3	Código-fonte	5
3.4	Análise	6
4	Questão 3: Busca em Árvore BST vs. Busca Binária em Vetor	6
4.1	Enunciado	6
4.2	Detalhes de Implementação	6
4.3	Análise Comparativa e Resultados Medidos	7
5	Questão 4: Árvore AVL vs. Árvore BST	8
5.1	Enunciado	8
5.2	Detalhes de Implementação	8
5.2.1	Árvore AVL – Inserção Iterativa com Pilha de Ancestrais	8
5.2.2	Cálculo de Altura	9
5.3	Resultados Obtidos	10
5.4	Análise Comparativa	11
5.4.1	Tempo de Criação	12
5.4.2	Altura das Árvores	12
5.4.3	Tempo de Busca	12
6	Organização dos Módulos TAD	13
7	Conclusão	13

1 Introdução e Metodologia

Este relatório detalha a implementação e a análise empírica das estruturas de dados baseadas em **árvores binárias**, conforme especificado no Trabalho 2 de AED2. O projeto foi inteiramente desenvolvido em linguagem C, seguindo rigorosamente o paradigma de **Tipos Abstratos de Dados (TAD)**, com separação entre interfaces (`.h`) e implementações (`.c`).

Os seguintes módulos TAD foram criados e/ou utilizados:

- **TAD-arvores** (`TAD-arvores.h` / `.c`) – Árvore Binária de Pesquisa (BST) com inserção recursiva, caminhamentos (pré-ordem, em-ordem, pós-ordem), busca, remoção e funções auxiliares.
- **TAD-pacotes** (`TAD-pacotes.h` / `.c`) – Árvore binária de pesquisa genérica para armazenar *pacotes* (compostos por um identificador inteiro e dados de texto), incluindo inserção, busca, travessia em-ordem e exportação para arquivo.
- **TAD-avl** (`TAD-avl.h` / `.c`) – Árvore AVL com inserção recursiva (com rotações simples e duplas para balanceamento), busca recursiva, cálculo de altura e liberação recursiva de memória.
- **TAD-Vetorzao** (`TAD-Vetorzao.h` / `.c`) – Vetor de 1.000.000 de inteiros com busca binária, Quicksort e funções de sorteio de chaves (reaproveitado do Trabalho 1).
- **TAD-tempo** (`TAD-tempo.h` / `.c`) – Funções para medição de tempo de execução usando `clock_gettime` com precisão de nanossegundos.

A medição de tempo foi realizada com `CLOCK_REALTIME` do POSIX, convertendo a diferença entre os *timestamps* para segundos.

2 Questão 1: Árvore Binária com Caminhamentos

2.1 Enunciado

Criar uma árvore binária de pesquisa com aproximadamente 20 elementos e mostrar o resultado para os três tipos de caminhamento: pré-fixado, central e pós-fixado.

2.2 Detalhes de Implementação

O programa `questao01.c` cria uma árvore binária de pesquisa e insere nela 20 números inteiros aleatórios (gerados com `rand() % 100`). Em seguida, executa os três caminhamentos clássicos:

- **Pré-ordem (Pré-fixado):** visita a raiz, depois a subárvore esquerda e, por fim, a subárvore direita. Ordem: *Raiz* $\rightarrow$ *Esquerda* $\rightarrow$ *Direita*.
- **Em-ordem (Central / Infixado):** visita a subárvore esquerda, depois a raiz e, por fim, a subárvore direita. Em uma BST, esse caminhamento produz os elementos em **ordem crescente**. Ordem: *Esquerda* $\rightarrow$ *Raiz* $\rightarrow$ *Direita*.
- **Pós-ordem (Pós-fixado):** visita a subárvore esquerda, depois a subárvore direita e, por fim, a raiz. Ordem: *Esquerda* $\rightarrow$ *Direita* $\rightarrow$ *Raiz*.

A inserção na BST segue o algoritmo recursivo implementado em `TAD-arvores.c`: se a chave é menor que o nó atual, insere à esquerda; caso contrário, insere à direita.

2.3 Código-fonte

```
1 #include <stdio.h>
2 #include <time.h>
3 #include <stdlib.h>
4 #include <locale.h>
5 #include <math.h>
6 #include "TAD-arvores.h"
7
8 int main()
9 {
10     setlocale(LC_ALL, "Portuguese");
11     Arvore *arv = criarArvore();
12
13     int i, n = 20;
14     srand(time(NULL));
15     for (i = 0; i < n; i++)
16     {
17         int num = rand() % 100;
18         printf("%d ", num);
19         arv = inserirNaArvore(arv, num);
20     }
21     printf("\n");
22     printf("Pre-ordem: ");
23     preOrdem(arv);
24     printf("\n");
25     printf("Em-ordem: ");
26     emOrdem(arv);
27     printf("\n");
28     printf("Pos-ordem: ");
29     posOrdem(arv);
30     printf("\n");
31
32     return 0;
33 }
```

Listing 1: questao01.c – Criação da BST e caminhamentos

2.4 Análise

O caminharmento **em-ordem** é particularmente relevante em árvores binárias de pesquisa, pois percorre os nós em ordem crescente de chave, ou seja, produz uma sequência ordenada dos elementos inseridos. Isso demonstra na prática a propriedade fundamental da BST: para todo nó, os elementos da subárvore esquerda possuem chave menor e os da subárvore direita possuem chave maior.

Os caminharmentos **pré-ordem** e **pós-ordem** são úteis, respectivamente, para serializar/reconstruir a árvore e para liberar memória de forma segura (liberando os filhos antes do pai).

A complexidade de cada caminharmento é $O(n)$, onde n é o número de nós da árvore, pois cada nó é visitado exatamente uma vez.

3 Questão 2: Simulação de Pacotes de Rede

3.1 Enunciado

Criar uma árvore binária de pesquisa para controlar o recebimento de pacotes em uma transmissão em rede. Cada pacote é identificado por um número inteiro e contém um dado do tipo texto. A ordem de chegada deve ser parcialmente ordenada e pode haver pacotes repetidos (simulando retransmissão). No final, o programa deve montar o arquivo com os dados na ordem correta.

3.2 Detalhes de Implementação

O programa `questao02.c` utiliza o TAD `TAD-pacotes` para criar uma árvore binária de pesquisa onde cada nó armazena um **pacote** composto por:

- `id` (inteiro) – identificador sequencial do pacote.
- `dados[100]` (string) – conteúdo de texto do pacote.

A estrutura `ArvorePacotes` é definida como:

```
1 typedef struct Pacote {
2     int id;
3     char dados[TAM_DADOS];
4 } Pacote;
5
6 typedef struct noPacote {
7     Pacote pacote;
8     struct noPacote *esq;
9     struct noPacote *dir;
10 } ArvorePacotes;
```

Listing 2: Estrutura do pacote e da árvore de pacotes

Os pacotes são inseridos na árvore com ordem de chegada parcialmente desordenada (ex.: pacotes 3, 1, 2, 4, 5, 6). A inserção utiliza o `id` como chave de ordenação. Pacotes com `id` duplicado são ignorados, simulando o descarte de retransmissões:

```
1 else {  
2     printf("Pacote duplicado ignorado: %d\n", p.id);  
3 }
```

Listing 3: Tratamento de pacotes duplicados na inserção

Ao final, o programa percorre a árvore **em-ordem** e grava o campo `dados` de cada pacote sequencialmente em um arquivo texto (`arquivo_final.txt`), reconstituindo a mensagem original na ordem correta.

3.3 Código-fonte

```
1 #include <stdio.h>  
2 #include "TAD-pacotes.h"  
3  
4 int main()  
5 {  
6     ArvorePacotes *raiz;  
7     raiz = criarArvorePacotes();  
8  
9     Pacote p1 = criarPacote(3, " ");  
10    Pacote p2 = criarPacote(1, "Wi-");  
11    Pacote p3 = criarPacote(2, "fi");  
12    Pacote p4 = criarPacote(4, "Ral");  
13    Pacote p5 = criarPacote(5, "ph!");  
14    Pacote p6 = criarPacote(6, " ");  
15  
16    Pacote p7 = gerarPacoteAutomatico(7);  
17  
18    raiz = inserirPacote(raiz, p1);  
19    raiz = inserirPacote(raiz, p2);  
20    raiz = inserirPacote(raiz, p3);  
21    raiz = inserirPacote(raiz, p4);  
22    raiz = inserirPacote(raiz, p5);  
23    raiz = inserirPacote(raiz, p6);  
24  
25    printf("\nPACOTES EM ORDEM:\n\n");  
26    emOrdemPacotes(raiz);  
27  
28    FILE *fp;  
29    fp = fopen("arquivo_final.txt", "w");  
30    salvarArquivo(raiz, fp);  
31    fclose(fp);  
32  
33    liberarArvorePacotes(raiz);  
34    return 0;  
35 }
```

Listing 4: questao02.c – Simulação de pacotes de rede

3.4 Análise

Esta questão demonstra uma aplicação prática de árvores binárias de pesquisa: a **reordenação de dados que chegam fora de sequência**. Na comunicação em rede (por exemplo, protocolo UDP), pacotes podem chegar desordenados ou duplicados. A BST permite:

1. **Inserção eficiente** com complexidade média $O(\log n)$.
2. **Descarte automático de duplicatas** durante a inserção (comparação pelo id).
3. **Reconstrução ordenada** via caminhamento em-ordem, que percorre os pacotes na sequência correta do identificador.

A função `salvarArquivo` realiza o caminhamento em-ordem e grava os dados de cada pacote no arquivo, reconstituindo a mensagem. No exemplo demonstrado, a mensagem final gravada em `arquivo_final.txt` é composta pela concatenação dos campos `dados` dos pacotes na ordem crescente de seus ids.

4 Questão 3: Busca em Árvore BST vs. Busca Binária em Vetor

4.1 Enunciado

Criar uma árvore binária de pesquisa com 1.000.000 de elementos inteiros. Comparar o tempo da busca na árvore com o tempo da busca binária no vetor (reaproveitando a implementação do Trabalho 1). Executar 30 buscas (pelo menos 15 com elementos presentes) e calcular a média dos tempos.

4.2 Detalhes de Implementação

O programa `questao03.c` realiza as seguintes etapas:

1. **Geração dos dados:** 1.000.000 de inteiros aleatórios são gerados e inseridos simultaneamente na BST (via `inserirNaArvore`) e no vetor (via `inserirNoVetor`).
2. **Ordenação do vetor:** O vetor é ordenado com Quicksort para viabilizar a busca binária.
3. **Busca com elementos existentes (15 execuções):** Sorteia-se uma chave *presente* no vetor (via `sortearElementoNoVetor`) e mede-se o tempo de busca na BST e da busca binária no vetor.
4. **Busca com elementos aleatórios (15 execuções):** Sorteia-se uma chave *aleatória* (via `sortearElemento`), que pode ou não existir nas estruturas, e mede-se os tempos analogamente.

5. **Cálculo da média:** Ao final, calcula-se a média dos 30 tempos de busca para cada estrutura.

```

1 for(int i=0; i<15; i++){
2     int chave = sortearElementoNoVetor();
3
4     clock_gettime(CLOCK_REALTIME, &inicio);
5     buscar(arv, chave);
6     clock_gettime(CLOCK_REALTIME, &fim);
7
8     tempoArv += tempoDeExecucao(inicio, fim);
9
10    clock_gettime(CLOCK_REALTIME, &inicio);
11    buscaBinaria(chave);
12    clock_gettime(CLOCK_REALTIME, &fim);
13
14    tempoVet += tempoDeExecucao(inicio, fim);
15 }

```

Listing 5: Trecho de questao03.c – Loop de busca com elementos existentes

4.3 Análise Comparativa e Resultados Medidos

Com a execução do programa na versão mais recente, obtivemos as seguintes medições práticas para 1.000.000 de elementos:

Tabela 1: Resultados práticos: Tempos de Busca e Consumo de Memória (1.000.000 elementos)

Estrutura	Tempo Médio de Busca (s)	Memória Alocada (Bytes)	Aprox. (MB)
Árvore BST	0,000003	24.000.000	22 MB
Busca Binária (Vetor)	0,000001	4.000.000	3 MB

Ambas as buscas possuem complexidade média teórica $O(\log n)$, porém existem diferenças práticas significativas evidenciadas pelos dados:

- **Desempenho de Busca:** A busca binária no vetor foi **3 vezes mais rápida** que a busca na BST (0,000001s vs 0,000003s). Isso ocorre porque o vetor possui excelente **localidade de referência** (dados contíguos na memória), aproveitando de forma muito mais eficiente a memória cache do processador, enquanto a árvore exige saltos de memória constantes ao seguir ponteiros.
- **Consumo de Memória:** A BST consumiu **6 vezes mais memória** que o vetor (24 MB vs 4 MB, uma diferença de aproximadamente 19 MB). No vetor, armazena-se apenas o inteiro (4 bytes por elemento). Na BST, cada nó exige armazenar o dado mais dois ponteiros (**esq** e **dir**), totalizando no mínimo 24 bytes por nó em sistemas de 64 bits (assumindo padding e alinhamento de 4 bytes de **info** e 8 bytes por ponteiro).

- **Custo de Manutenção:** O vetor precisou ser previamente ordenado (custo $O(n \log n)$ via Quicksort) para viabilizar a busca rápida, enquanto a BST organiza as chaves dinamicamente a cada inserção.

5 Questão 4: Árvore AVL vs. Árvore BST

5.1 Enunciado

Criar uma árvore AVL e uma árvore binária de pesquisa com os mesmos 1.000.000 de elementos inteiros. Criar 10 árvores de cada tipo (com os mesmos valores) e comparar: (a) o tempo de criação, (b) a altura das árvores em cada execução, e (c) o tempo de busca (30 consultas) com cálculo da média.

5.2 Detalhes de Implementação

O programa `questao04.c` é o mais completo do trabalho. Suas características principais são:

1. **Parâmetros fixos:** $n = 1.000.000$ elementos, 10 execuções, 30 consultas por execução.
2. **Semente fixa para dados:** `srand(42)` garante reprodutibilidade — os mesmos 1.000.000 de números são usados em todas as 10 execuções.
3. **Tratamento uniforme de duplicatas:** Tanto a BST quanto a AVL **rejeitam chaves duplicadas**, garantindo que ambas armazenam exatamente o mesmo conjunto de elementos distintos para uma comparação justa.
4. **Chaves de busca mistas:** 15 chaves são selecionadas dentre os elementos inseridos (garantia de existência) e 15 são geradas aleatoriamente (possivelmente inexistentes).

5.2.1 Árvore AVL – Inserção Recursiva com Rotações

A implementação da AVL em `TAD-avl.c` utiliza uma abordagem **recursiva** para inserção. Após inserir o novo nó na posição correta da BST, cada chamada recursiva, ao retornar, atualiza a altura do nó ancestral e verifica o fator de balanceamento, aplicando rotações quando necessário:

```
1 AVL* inserirNaAVL(AVL* arv, int chave) {
2     if (arv == NULL) {
3         arv = (AVL*) malloc(sizeof(AVL));
4         arv->info = chave;
5         arv->altura = 1;
6         arv->esq = NULL;
7         arv->dir = NULL;
8         return arv;
9     }
```

```

10     if (chave < arv->info)
11         arv->esq = inserirNaAVL(arv->esq, chave);
12     else if (chave > arv->info)
13         arv->dir = inserirNaAVL(arv->dir, chave);
14     else
15         return arv; /* Chave duplicada: ignora */
16
17     atualizarAltura(arv);
18     int fb = fatorBalanceamento(arv);
19
20     if (fb > 1 && chave < arv->esq->info)
21         return rotacaoDireita(arv);
22     if (fb < -1 && chave > arv->dir->info)
23         return rotacaoEsquerda(arv);
24     if (fb > 1 && chave > arv->esq->info)
25         return rotacaoEsquerdaDireita(arv);
26     if (fb < -1 && chave < arv->dir->info)
27         return rotacaoDireitaEsquerda(arv);
28
29     return arv;
30 }

```

Listing 6: Inserção recursiva na AVL com balanceamento

As rotações implementadas são:

- **Rotação Simples à Direita** (`rotacaoDireita`) – corrige desbalanceamento esquerda-esquerda.
- **Rotação Simples à Esquerda** (`rotacaoEsquerda`) – corrige desbalanceamento direita-direita.
- **Rotação Dupla Esquerda-Direita** (`rotacaoEsquerdaDireita`) – combinação de rotação à esquerda na subárvore esquerda seguida de rotação à direita no nó.
- **Rotação Dupla Direita-Esquerda** (`rotacaoDireitaEsquerda`) – combinação de rotação à direita na subárvore direita seguida de rotação à esquerda no nó.

5.2.2 BST – Inserção Iterativa Local

Para a BST, o programa utiliza uma função local `inserirBST` com inserção **iterativa** que também rejeita duplicatas, garantindo paridade com a AVL:

```

1 Arvore* inserirBST(Arvore* arv, int chave) {
2     Arvore* novo = (Arvore*)malloc(sizeof(Arvore));
3     novo->info = chave;
4     novo->esq = NULL;
5     novo->dir = NULL;
6     if (arv == NULL) return novo;
7
8     Arvore* atual = arv;
9     while (1) {
10         if (chave < atual->info) {
11             if (atual->esq == NULL) { atual->esq = novo; return arv; }
12             atual = atual->esq;

```

```

13     } else if (chave > atual->info) {
14         if (atual->dir == NULL) { atual->dir = novo; return arv; }
15         atual = atual->dir;
16     } else {
17         free(novo); /* Duplicata: ignora */
18         return arv;
19     }
20 }
21 }

```

Listing 7: Inserção iterativa na BST (rejeita duplicatas)

5.2.3 Cálculo de Altura

A altura da BST é calculada via função recursiva local (`alturaBST`), que retorna $1 + \max(h_{\text{esq}}, h_{\text{dir}})$. A altura da AVL é obtida diretamente do campo `altura` armazenado no nó raiz (`alturaRealAVL`), pois a AVL mantém essa informação atualizada a cada inserção.

5.3 Resultados Obtidos

Tabela 2: Tempo de criação e altura das árvores (10 execuções, 1.000.000 elementos)

Execução	Tempo Criação BST (s)	Tempo Criação AVL (s)	Altura BST	Altura AVL
1	0,721448	1,397104	49	23
2	0,626516	1,138523	49	23
3	0,619249	1,150847	49	23
4	0,616477	1,144245	49	23
5	0,627239	1,158559	49	23
6	0,713685	1,141972	49	23
7	0,615231	1,144061	49	23
8	0,623877	1,153420	49	23
9	0,624783	1,154023	49	23
10	0,611095	1,169304	49	23
Média	0,639960	1,175206	49	23

Tabela 3: Tempo de busca – BST vs. AVL (30 consultas $\times$ 10 execuções)

Métrica	BST (s)	AVL (s)
Tempo total (30 consultas $\times$ 10 exec.)	0,000468	0,000647
Tempo médio por consulta	0,000001560	0,000002156

5.4 Análise Comparativa

Tabela 4: Comparação teórica: BST vs. AVL

Característica	BST	AVL
Inserção (caso médio)	$O(\log n)$	$O(\log n)$
Inserção (pior caso)	$O(n)$	$O(\log n)$
Busca (caso médio)	$O(\log n)$	$O(\log n)$
Busca (pior caso)	$O(n)$	$O(\log n)$
Altura medida (10^6 elementos aleatórios)	49	23
Custo adicional por inserção	Nenhum	Cálculo de FB + rotações
Memória por nó	info + 2 ponteiros	info + altura + 2 ponteiros

5.4.1 Tempo de Criação

Conforme os resultados obtidos, a criação da **AVL foi cerca de $1,84\times$ mais lenta** que a da BST (média de 1,175s contra 0,640s), ou seja, a BST foi **45,5% mais rápida**. Isso ocorre porque cada inserção na AVL envolve:

- Cálculo do fator de balanceamento ($FB = h_{\text{esq}} - h_{\text{dir}}$).
- Atualização da altura do nó.
- Eventual execução de rotações (simples ou duplas) para manter $|FB| \leq 1$.

Na BST simples, a inserção iterativa é direta: basta descer até encontrar a posição correta e alocar o nó, sem custos adicionais de balanceamento.

5.4.2 Altura das Árvores

A diferença de altura é a vantagem mais significativa da AVL. Os resultados mediram:

- **BST:** altura = **49** (constante em todas as 10 execuções, pois a mesma semente é usada). Isso equivale a $\approx 2,45 \cdot \log_2(10^6)$, condizente com a altura esperada de uma BST com dados aleatórios.
- **AVL:** altura = **23** (constante em todas as execuções). Isso é consistente com o limite teórico de $1,44 \cdot \log_2(n + 2) \approx 28,7$, e demonstra que a AVL mantém altura muito próxima do ótimo $\lceil \log_2 n \rceil \approx 20$.

A AVL apresentou altura **53% menor** que a BST (23 vs. 49), o que se traduz em menos comparações durante a busca. Essa garantia de altura logarítmica assegura **busca consistentemente rápida**, independentemente da ordem de inserção.

5.4.3 Tempo de Busca

Os resultados de busca mostraram que a BST foi ligeiramente mais rápida na média por consulta (**0,000001560s** contra **0,000002156s** da AVL), com a BST sendo **27,6% mais rápida**. Embora a AVL possua altura menor, a BST apresentou vantagem neste cenário específico. Isso pode ser atribuído a:

- A BST utiliza inserção **iterativa**, resultando em uma disposição de nós na memória potencialmente mais favorável à cache do processador.
- Com dados aleatórios, a BST não degenera significativamente, mantendo profundidade média de busca competitiva.
- O *overhead* da recursão na busca da AVL (empilhamento de frames de chamada) pode introduzir custo adicional em buscas individuais ultrarrápidas.

É fundamental destacar que, com dados aleatórios, a BST apresentou desempenho competitivo. No entanto, a AVL oferece a **garantia** de que esse desempenho se mantém mesmo com dados adversos (inserções em ordem crescente, por exemplo), enquanto a BST degeneraria para $O(n)$. A escolha entre as estruturas depende do cenário de uso: se os dados são imprevisíveis ou potencialmente ordenados, a AVL é a escolha segura.

6 Organização dos Módulos TAD

A tabela a seguir resume a organização modular do projeto:

Tabela 5: Módulos TAD do projeto

Módulo	Arquivos	Responsabilidade
TAD-arvores	.h / .c	BST genérica (int): inserção, busca, remoção, caminhamentos
TAD-pacotes	.h / .c	BST para pacotes (id + dados): inserção, busca, exportação
TAD-avl	.h / .c	AVL recursiva: inserção com rotações, busca, altura
TAD-Vetorzao	.h / .c	Vetor de 1M inteiros, busca binária, Quicksort
TAD-tempo	.h / .c	Medição de tempo via <code>clock_gettime</code>

7 Conclusão

Os experimentos realizados neste trabalho permitem as seguintes conclusões:

1. **Questão 1 – Caminhamentos:** Os três caminhamentos (pré-ordem, em-ordem, pós-ordem) demonstram as diferentes formas de percorrer uma árvore binária, sendo o caminhamento em-ordem especialmente importante por produzir a sequência ordenada dos elementos em uma BST.

2. **Questão 2 – Pacotes de rede:** A BST se mostrou uma estrutura eficaz para reordenar dados que chegam fora de sequência, com a vantagem adicional de descartar duplicatas de forma natural durante a inserção. O caminhar em ordem permite reconstituir a mensagem original.
3. **Questão 3 – BST vs. Busca Binária:** Ambas as abordagens possuem complexidade média $O(\log n)$, porém a busca binária no vetor apresenta vantagens práticas de localidade de cache. Por outro lado, a BST não requer ordenação prévia, sendo mais adequada para cenários dinâmicos de inserção e busca.
4. **Questão 4 – AVL vs. BST:** A AVL oferece garantia de desempenho logarítmico no pior caso, à custa de um tempo de criação cerca de 84% superior. Com dados aleatórios, a BST apresentou busca ligeiramente mais rápida devido à localidade de cache e ausência de overhead recursivo na busca. No entanto, para aplicações que priorizam a **consistência** das operações (especialmente com dados não aleatórios), a AVL é a escolha superior. A BST simples é preferível quando o custo de inserção é crítico e os dados são suficientemente aleatórios para evitar degeneração.

Os dados empíricos confirmam as previsões teóricas da análise de complexidade assintótica, reforçando a importância de escolher a estrutura de dados adequada conforme as características do problema e dos dados de entrada.